

Cahier des Charges : MedCare Connect - Marketplace Médicale au Sénégal

- **Version** : 1.0.0
 - **Date** : 26 Mai 2024
 - **Auteur** : Aboubacar SAWADO
-

1. Introduction

1.1. Contexte du Projet

Le système de santé au Sénégal fait face à plusieurs défis, notamment le manque de visibilité sur l'offre de services médicaux et paramédicaux, la complexité de l'estimation des coûts, et la difficulté de vérifier la prise en charge par les assurances. De plus, la communication entre patients et prestataires de santé peut souvent être fragmentée et inefficace.

MedCare Connect se positionne comme une solution innovante pour centraliser et simplifier l'accès aux services de santé au Sénégal. Il s'agit d'une plateforme web interactive de type "marketplace" qui met en relation les patients avec divers organismes de santé (hôpitaux, cliniques, laboratoires, pharmacies, centres d'imagerie, etc.).

L'objectif principal est d'offrir une transparence et une facilité d'accès aux informations vitales pour les patients, tout en fournissant aux prestataires un outil efficace pour gérer leur visibilité, leurs services et leurs interactions.

1.2. Objectifs Généraux

- **Améliorer l'accessibilité à l'information santé** pour les patients au Sénégal.
- **Simplifier le processus de recherche et d'estimation des coûts** pour les examens et actes médicaux.
- **Faciliter la vérification de la prise en charge des assurances** par les différents prestataires.
- **Optimiser la gestion et la visibilité des services** offerts par les organismes de santé.
- **Établir un canal de communication direct et efficace** entre patients et prestataires.
- **Construire une application robuste, sécurisée et évolutive**, prête à être déployée.
- **Proposer une expérience utilisateur moderne, épurée et intuitive.**

1.3. Public Cible

- **Patients / Clients** : Individus recherchant des services médicaux, souhaitant obtenir des informations sur les coûts et les assurances, et contacter des prestataires.
 - **Organismes de Santé (Prestataires)** : Hôpitaux, cliniques, laboratoires d'analyses, centres d'imagerie, pharmacies, centres de rééducation, cabinets spécialisés, etc., souhaitant digitaliser leur offre et améliorer leur visibilité.
-

2. Description Fonctionnelle

MedCare Connect sera composé de deux interfaces principales : une pour les patients et une pour les organismes de santé, chacune avec des fonctionnalités spécifiques.

2.1. Fonctionnalités Communes

- **Gestion des Comptes Utilisateurs** :
 - Inscription et Connexion (Patient / Prestataire).
 - Réinitialisation de mot de passe.
 - Gestion du profil (informations personnelles, contact).

2.2. Fonctionnalités pour les Patients (Clients)

1. **Recherche de Services Médicaux** :
 - Recherche par type de service (Biologie Médicale, Imagerie, etc.).
 - Recherche par acte spécifique (NFS, Radiographie Thorax, etc.).
 - Recherche par nom d'organisme de santé.
 - Recherche par localisation géographique.
 - Filtres avancés : par assurance acceptée, par disponibilité, par prix.
2. **Visualisation des Informations des Prestataires** :
 - Profil détaillé du prestataire : nom, adresse, contacts, description, horaires, liste des services/actes proposés.
 - Prix indicatifs des actes.
 - Liste des assurances acceptées par le prestataire.
3. **Panier et Estimation des Coûts** :
 - Ajout d'actes médicaux de différents prestataires dans un "panier".
 - Visualisation du récapitulatif du panier avec les prix indicatifs.
 - Sélection de son assurance pour voir une estimation de la prise en charge (calcul basé sur le taux fourni par le prestataire).
 - Génération d'un devis estimatif (non contractuel).
4. **Communication avec les Prestataires** :
 - Fonctionnalité "Contacter via WhatsApp" directement depuis la plateforme, pré-remplissant un message avec la demande du patient et les informations du panier si applicable.
 - Système de messagerie interne à la plateforme (future évolution pour la gestion de RDV).
5. **Gestion du Profil Patient** :
 - Visualisation de l'historique des recherches et des paniers sauvegardés.

- Gestion de ses informations personnelles.

2.3. Fonctionnalités pour les Organismes de Santé (Prestataires)

1. **Tableau de Bord Administratif :**
 - Accès à un tableau de bord intuitif après connexion.
 - Vue d'ensemble de l'activité (nombre de consultations de profil, demandes de contact, etc.).
 2. **Gestion du Profil de l'Organisme :**
 - Renseigner les informations de base : nom, adresse, numéro de téléphone, email, site web, description.
 - Télécharger des photos / logos.
 - Définir les horaires d'ouverture.
 3. **Configuration des Services et Actes Médicaux :**
 - Ajout, modification et suppression des ServiceMedical et ActeMedical proposés.
 - Définition des prix indicatifs pour chaque acte.
 - Possibilité de catégoriser les actes selon la structure hiérarchique définie (Niveau 1, Niveau 2, Niveau 3).
 4. **Gestion des Assurances et Prise en Charge :**
 - Sélection des Assurance acceptées par l'organisme.
 - Définition du taux de prise en charge pour chaque assurance, soit globalement pour l'établissement, soit spécifiquement pour certains actes.
 5. **Gestion des Demandes de Contact :**
 - Visualisation des messages reçus des patients (via WhatsApp ou messagerie interne future).
-

3. User Stories

Voici une liste détaillée de user stories pour clarifier les attentes fonctionnelles du point de vue de l'utilisateur.

3.1. User Stories Communes

- **US-COMMON-001 (Inscription)**
 - **En tant que** nouvel utilisateur,
 - **Je veux** pouvoir m'inscrire sur la plateforme,
 - **Afin de** créer mon compte (Patient ou Prestataire).
- **US-COMMON-002 (Connexion)**
 - **En tant que** utilisateur enregistré,
 - **Je veux** pouvoir me connecter à la plateforme avec mon email/mot de passe,
 - **Afin de** d'accéder à mon espace personnel.
- **US-COMMON-003 (Réinitialisation Mot de Passe)**
 - **En tant que** utilisateur ayant oublié mon mot de passe,
 - **Je veux** pouvoir le réinitialiser via mon adresse email,

- **Afin de** retrouver l'accès à mon compte.
- **US-COMMON-004 (Gestion Profil)**
 - **En tant que** utilisateur,
 - **Je veux** pouvoir modifier les informations de mon profil (nom, contact, etc.),
 - **Afin de** maintenir mes informations à jour.

3.2. User Stories pour les Patients (Clients)

- **US-PATIENT-001 (Recherche par Service)**
 - **En tant que** patient,
 - **Je veux** pouvoir rechercher un service médical par sa catégorie (ex: "Biologie Médicale"),
 - **Afin de** trouver des prestataires proposant ce type de service.
- **US-PATIENT-002 (Recherche par Acte Spécifique)**
 - **En tant que** patient,
 - **Je veux** pouvoir rechercher un acte médical spécifique (ex: "NFS", "Radiographie Thorax"),
 - **Afin de** trouver les prestataires qui le réalisent.
- **US-PATIENT-003 (Recherche par Nom de Prestataire)**
 - **En tant que** patient,
 - **Je veux** pouvoir rechercher un organisme de santé par son nom,
 - **Afin de** trouver ses informations et les services qu'il propose.
- **US-PATIENT-004 (Recherche par Localisation)**
 - **En tant que** patient,
 - **Je veux** pouvoir filtrer les résultats de recherche par localisation géographique,
 - **Afin de** trouver des prestataires proches de chez moi.
- **US-PATIENT-005 (Filtre par Assurance)**
 - **En tant que** patient,
 - **Je veux** pouvoir filtrer les prestataires par les assurances qu'ils acceptent,
 - **Afin de** trouver ceux qui sont compatibles avec ma couverture.
- **US-PATIENT-006 (Voir Profil Prestataire)**
 - **En tant que** patient,
 - **Je veux** pouvoir consulter le profil détaillé d'un organisme de santé,
 - **Afin de** connaître ses contacts, horaires, adresse et tous les services qu'il offre.
- **US-PATIENT-007 (Voir Prix Indicatifs)**
 - **En tant que** patient,
 - **Je veux** voir les prix indicatifs des actes médicaux proposés par un prestataire,
 - **Afin de** me faire une idée du coût avant de me déplacer.
- **US-PATIENT-008 (Ajouter au Panier)**
 - **En tant que** patient,
 - **Je veux** pouvoir ajouter un ou plusieurs actes médicaux à un "panier",
 - **Afin de** regrouper mes besoins et obtenir une estimation globale.
- **US-PATIENT-009 (Estimer Coût Total Panier)**
 - **En tant que** patient,
 - **Je veux** visualiser le coût total indicatif des actes dans mon panier,

- **Afin de** planifier mon budget santé.
- **US-PATIENT-010 (Simuler Prise en Charge Assurance)**
 - **En tant que** patient,
 - **Je veux** pouvoir sélectionner mon assurance dans le panier,
 - **Afin de** voir une estimation du montant restant à ma charge après la prise en charge de l'assurance.
- **US-PATIENT-011 (Contacter via WhatsApp)**
 - **En tant que** patient,
 - **Je veux** pouvoir envoyer un message pré-rempli à un prestataire via WhatsApp directement depuis la plateforme,
 - **Afin de** demander des informations ou confirmer la disponibilité d'un acte de mon panier.
- **US-PATIENT-012 (Historique Panier)**
 - **En tant que** patient,
 - **Je veux** pouvoir consulter l'historique de mes paniers et recherches,
 - **Afin de** retrouver facilement des informations antérieures.

3.3. User Stories pour les Organismes de Santé (Prestataires)

- **US-PRESTA-001 (Accéder Tableau de Bord)**
 - **En tant que** prestataire,
 - **Je veux** accéder à un tableau de bord clair après connexion,
 - **Afin de** gérer mon profil et mes services.
- **US-PRESTA-002 (Gérer Profil Organisme)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir ajouter ou modifier les informations de mon organisme (adresse, contacts, description, photos, horaires),
 - **Afin de** présenter mon établissement de manière complète et à jour.
- **US-PRESTA-003 (Ajouter Service Médical)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir ajouter un nouveau service médical (ex: "Biologie Médicale") à mon offre,
 - **Afin de** lister tous les types de services que je propose.
- **US-PRESTA-004 (Ajouter Acte Médical)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir ajouter un acte médical spécifique (ex: "NFS / Hémogramme") sous un service existant,
 - **Afin de** détailler l'ensemble des prestations que je réalise.
- **US-PRESTA-005 (Définir Prix Acte)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir définir le prix indicatif pour chaque acte médical que je propose,
 - **Afin de** permettre aux patients d'obtenir des estimations de coût.
- **US-PRESTA-006 (Gérer Assurances Acceptées)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir sélectionner les compagnies d'assurance avec lesquelles j'ai des conventions,
 - **Afin de** informer les patients sur ma prise en charge.

- **US-PRESTA-007 (Définir Taux Prise en Charge)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir définir un taux de prise en charge pour chaque assurance acceptée (globalement ou par acte si nécessaire),
 - **Afin de** fournir des estimations précises aux patients.
 - **US-PRESTA-008 (Voir Demandes de Contact)**
 - **En tant que** prestataire,
 - **Je veux** pouvoir visualiser les demandes de contact ou les messages envoyés par les patients,
 - **Afin de** pouvoir y répondre rapidement.
-

4. Modèle de Données (Django Models)

Cette section décrit les modèles de données envisagés pour le backend Django.

code Python

users/models.py

from django.db import models

from django.contrib.auth.models import AbstractUser

class User(AbstractUser):

 USER_TYPE_CHOICES = (

 ("patient", "Patient"),

 ("prestataire", "Prestataire de Santé"),

)

 user_type = models.CharField(max_length=20, choices=USER_TYPE_CHOICES, default="patient")

 phone_number = models.CharField(max_length=20, blank=True, null=True)

 def __str__(self):

 return self.username

healthcare/models.py

from django.db import models

from users.models import User

class OrganismeDeSante(models.Model):

 user = models.OneToOneField(User, on_delete=models.CASCADE,

 related_name="healthcare_provider_profile")

 name = models.CharField(max_length=255)

 address = models.CharField(max_length=255)

 city = models.CharField(max_length=100, default="Dakar") # Ou un champ plus générique pour le Sénégal

 contact_email = models.EmailField(blank=True, null=True)

 contact_phone = models.CharField(max_length=20, blank=True, null=True)

```

description = models.TextField(blank=True, null=True)
website = models.URLField(blank=True, null=True)
opening_hours = models.JSONField(blank=True, null=True) # Ex: {"Mon": "08:00-17:00",
...}
logo = models.ImageField(upload_to="logos/", blank=True, null=True)
photos = models.ManyToManyField('Photo', blank=True) # Pour des photos additionnelles
is_active = models.BooleanField(default=False) # Pour approbation admin

def __str__(self):
    return self.name

class Photo(models.Model):
    image = models.ImageField(upload_to="organism_photos/")
    caption = models.CharField(max_length=255, blank=True)

    def __str__(self):
        return self.caption or "Photo"

class ServiceMedical(models.Model):
    # Correspond au Niveau 1 de votre document (e.g., "BIOLOGIE MÉDICALE")
    name = models.CharField(max_length=255, unique=True)
    description = models.TextField(blank=True, null=True)
    icon = models.CharField(max_length=50, blank=True, null=True) # Pour une icône
visuelle

    def __str__(self):
        return self.name

class ActeMedical(models.Model):
    # Peut être un Niveau 2 (e.g., "Hématologie") ou un Niveau 3 (e.g., "NFS /
Hémogramme")
    SERVICE_LEVEL_CHOICES = (
        (1, "Service Principal"), # Utilisé si on veut avoir des actes directement sous un
ServiceMedical
        (2, "Sous-Service"),
        (3, "Acte Spécifique"),
    )
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True, null=True)
    parent_service = models.ForeignKey('self', on_delete=models.SET_NULL, null=True,
blank=True, related_name='sub_acts') # Pour la hiérarchie N2/N3
    service_medical_category = models.ForeignKey(ServiceMedical,
on_delete=models.CASCADE, related_name='acts') # Lien vers Niveau 1
    level = models.IntegerField(choices=SERVICE_LEVEL_CHOICES, default=3) # Niveau
hiérarchique
    # Un acte peut avoir un prix différent selon l'organisme
    # Le prix_indicatif ici est un prix "moyen" ou "recommandé", le prestataire définira le sien.

```

```

class Meta:
    unique_together = ('name', 'parent_service', 'service_medical_category') # Empêcher
les doublons logiques

    def __str__(self):
        return f"{self.name} ({self.get_level_display()})"

class PrestataireActe(models.Model):
    # Lien Many-to-Many entre OrganismeDeSante et ActeMedical avec des attributs
supplémentaires
    organisme = models.ForeignKey(OrganismeDeSante, on_delete=models.CASCADE)
    acte = models.ForeignKey(ActeMedical, on_delete=models.CASCADE)
    price = models.DecimalField(max_digits=10, decimal_places=2) # Prix spécifique de l'acte
chez ce prestataire
    is_available = models.BooleanField(default=True)

    class Meta:
        unique_together = ('organisme', 'acte') # Un prestataire ne peut avoir le même acte
qu'une fois

    def __str__(self):
        return f"{self.organisme.name} - {self.acte.name} ({self.price} XOF)"

class Assurance(models.Model):
    name = models.CharField(max_length=255, unique=True)
    description = models.TextField(blank=True, null=True)
    logo = models.ImageField(upload_to="insurance_logos/", blank=True, null=True)

    def __str__(self):
        return self.name

class PriseEnChargeAssurance(models.Model):
    organisme = models.ForeignKey(OrganismeDeSante, on_delete=models.CASCADE)
    assurance = models.ForeignKey(Assurance, on_delete=models.CASCADE)
    # Taux de prise en charge applicable à TOUS les actes du prestataire pour cette
assurance
    # Sauf si un PriseEnChargeActeAssurance spécifique existe
    global_coverage_rate = models.DecimalField(max_digits=5, decimal_places=2,
default=0.00) # Ex: 0.80 pour 80%

    class Meta:
        unique_together = ('organisme', 'assurance')

    def __str__(self):
        return f"{self.organisme.name} - {self.assurance.name}
({self.global_coverage_rate*100}%)"

class PriseEnChargeActeAssurance(models.Model):

```



```

# Permet de définir un taux de prise en charge spécifique pour un ACTE donné
# Si cet enregistrement existe, il override le global_coverage_rate de
PriseEnChargeAssurance
    prise_en_charge_generale = models.ForeignKey(PriseEnChargeAssurance,
on_delete=models.CASCADE)
    acte = models.ForeignKey(ActeMedical, on_delete=models.CASCADE)
    coverage_rate = models.DecimalField(max_digits=5, decimal_places=2) # Ex: 0.90 pour
90%

class Meta:
    unique_together = ('prise_en_charge_generale', 'acte')

    def __str__(self):
        return f"{self.prise_en_charge_generale.organisme.name} - {self.acte.name} -
{self.prise_en_charge_generale.assurance.name} ({self.coverage_rate*100}%)"

class Cart(models.Model):
    patient = models.OneToOneField(User, on_delete=models.CASCADE,
related_name="cart")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    selected_insurance = models.ForeignKey(Assurance, on_delete=models.SET_NULL,
null=True, blank=True)

    def __str__(self):
        return f"Panier de {self.patient.username}"

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE, related_name="items")
    prestataire_acte = models.ForeignKey(PrestataireActe, on_delete=models.CASCADE) #
L'acte avec son prix chez un prestataire
    quantity = models.PositiveIntegerField(default=1)

    def __str__(self):
        return f"{self.quantity} x {self.prestataire_acte.acte.name} chez
{self.prestataire_acte.organisme.name}"

class Message(models.Model):
    sender = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="sent_messages")
    receiver = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="received_messages")
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)
    is_read = models.BooleanField(default=False)
    # Pour la fonctionnalité WhatsApp initiale
    # Un "type" pour distinguer les messages internes des liens WhatsApp générés
    MESSAGE_TYPE_CHOICES = (

```

```

        ("internal", "Message Interne"),
        ("whatsapp_request", "Demande WhatsApp"),
    )
    message_type = models.CharField(max_length=20,
    choices=MESSAGE_TYPE_CHOICES, default="internal")

    def __str__(self):
        return f"De {self.sender.username} à {self.receiver.username} ({self.timestamp.strftime('%Y-%m-%d %H:%M')})"

```

Explications des Relations :

- User est la base. OrganismeDeSante a un OneToOneField avec User pour les profils prestataires.
 - ServiceMedical représente les grandes catégories (Niveau 1).
 - ActeMedical représente les sous-catégories (Niveau 2) et les actes spécifiques (Niveau 3). La relation parent_service permet de construire la hiérarchie. service_medical_category assure le lien avec le Niveau 1.
 - PrestataireActe permet de lier un OrganismeDeSante à un ActeMedical et de spécifier le price pour cet acte chez ce prestataire. C'est crucial car les prix peuvent varier.
 - Assurance liste les compagnies.
 - PriseEnChargeAssurance définit le taux de prise en charge global d'une assurance par un organisme.
 - PriseEnChargeActeAssurance permet de *surcharger* ce taux global pour un ActeMedical spécifique.
 - Cart et CartItem gèrent le panier du patient, liant aux PrestataireActe pour référencer le prix spécifique.
 - Message gère la communication.
-

5. Architecture Technique

5.1. Backend

- **Langage** : Python 3.x
- **Framework Web** : Django (version stable et récente)
- **Base de Données** : PostgreSQL (recommandé pour la production pour sa robustesse et ses fonctionnalités avancées). SQLite pour le développement local.
- **API (si nécessaire)** : Django REST Framework (pour les interactions frontend/backend si une approche SPA est choisie, ou pour des intégrations futures).
- **Gestion des Médias** : Django's ImageField et FileField avec un stockage configuré (locale en dev, S3 ou équivalent en prod).

5.2. Frontend

- **Langages** : HTML5, CSS3, JavaScript (ES6+)
- **Framework CSS** : Tailwind CSS (pour un design utility-first, épuré et moderne).
- **Bibliothèque JS (si nécessaire)** : Alpine.js ou Vanilla JS pour des interactivités légères, ou React/Vue.js si l'application tend vers une SPA. **Pour ce projet, une approche Django templating avec Alpine.js/Vanilla JS est privilégiée pour la simplicité et la performance.**
- **Build Tool** : Webpack ou Vite (configuré via Django-Tailwind ou django-compressor pour la compilation et l'optimisation des assets Tailwind/JS).

5.3. Déploiement

- **Conteneurisation** : Docker et Docker Compose (pour faciliter le déploiement et l'environnement de développement).
- **Serveur Web** : Nginx (pour servir les requêtes et les fichiers statiques/médias).
- **Serveur d'Applications Python** : Gunicorn (pour faire le lien entre Nginx et Django).
- **Cloud Provider** : À définir (ex: AWS, Google Cloud, DigitalOcean) pour la production.

5.4. Gestion de Version

- **Git** : Utilisation de Git pour le contrôle de version, avec un dépôt hébergé sur GitHub ou GitLab.
-

6. Exigences Non Fonctionnelles

6.1. Performance

- Temps de chargement des pages inférieur à 3 secondes pour la majorité des requêtes.
- Temps de réponse de l'API inférieur à 500 ms pour 90% des requêtes critiques.
- Gestion efficace des requêtes concurrentes (scalabilité horizontale à prévoir).

6.2. Sécurité

- **Authentification et Autorisation** : Protection contre les attaques par force brute (limitation des tentatives de connexion). Gestion des rôles (patient, prestataire).
- **Protection des Données** : Hachage des mots de passe. Chiffrement des données sensibles en transit (HTTPS obligatoire).
- **Vulnérabilités Web** : Protection contre les injections SQL, XSS, CSRF (grâce aux mécanismes intégrés de Django).
- **Accès aux Données** : Les patients ne peuvent accéder qu'à leurs propres informations. Les prestataires n'accèdent qu'aux données de leur propre organisme.
- **Backups** : Stratégie de sauvegarde de la base de données et des médias régulière et automatisée.

6.3. Ergonomie et Design (Design Épuré, Pro et Moderne)

- **Interface Utilisateur (UI) :**
 - **Simplicité** : Minimiser le désordre visuel, se concentrer sur l'essentiel.
 - **Cohérence** : Utilisation d'un design system avec des composants réutilisables pour assurer une uniformité visuelle.
 - **Typographie** : Choix de polices modernes et lisibles (ex: Inter, Poppins ou équivalent sans-serif).
 - **Couleurs** : Palette de couleurs limitée et harmonieuse, privilégiant les tons neutres avec quelques touches de couleur pour les éléments interactifs ou importants (inspiration de couleurs de santé/technologie).
 - **Espacement** : Utilisation généreuse de l'espace blanc pour améliorer la lisibilité et l'expérience.
 - **Iconographie** : Icônes pertinentes et au style uniforme (ex: Feather Icons, Heroicons).
- **Expérience Utilisateur (UX) :**
 - **Intuitivité** : Navigation facile et logique. Les utilisateurs doivent pouvoir accomplir leurs tâches sans effort excessif.
 - **Clarté** : Messages d'erreur et de succès clairs et utiles.
 - **Feedback Visuel** : Animations subtiles et retours visuels pour les interactions (clics, survol).
 - **Accessibilité** : Prise en compte des normes d'accessibilité (contraste des couleurs, balisage sémantique).

6.4. Évolutivité

- L'architecture doit permettre l'ajout futur de fonctionnalités (gestion de RDV, paiements en ligne, évaluations, notifications push, etc.) sans refonte majeure.
- Possibilité de gérer un nombre croissant d'utilisateurs et de données.

6.5. Maintenabilité

- Code propre, commenté et conforme aux standards de Django et Python (PEP8).
- Tests unitaires et d'intégration pour les fonctionnalités critiques.
- Documentation technique à jour.

6.6. Compatibilité

- **Navigateurs Web** : Compatible avec les versions modernes des navigateurs les plus utilisés (Chrome, Firefox, Safari, Edge).
- **Responsive Design** : L'application doit être entièrement responsive et offrir une expérience optimale sur les ordinateurs de bureau, les tablettes et les smartphones.

7. Spécifications de Sécurité Détaillées

7.1. Protection des Données Personnelles et de Santé

- **Données Hachées** : Les mots de passe des utilisateurs doivent être stockés hachés avec des algorithmes robustes (Django utilise par défaut PBKDF2WithHMACSHA256).
- **Confidentialité** : Toutes les communications entre le client (navigateur) et le serveur doivent être chiffrées via HTTPS (certificats SSL/TLS).
- **Accès Limité** : Seuls les utilisateurs authentifiés et autorisés peuvent accéder aux données sensibles.
- **Pseudonymisation/Anonymisation** : Envisager pour les données analytiques non essentielles si elles peuvent révéler des informations personnelles sur la santé.
- **Conformité** : S'assurer de la conformité avec les réglementations locales sur la protection des données au Sénégal (si existantes et applicables) et les bonnes pratiques internationales.

7.2. Sécurité des Applications Web (OWASP Top 10)

- **A01:2021-Broken Access Control** :
 - Implémenter des vérifications d'autorisation strictes côté serveur pour toutes les requêtes (chaque utilisateur ne peut accéder qu'à ses propres données ou aux données publiques).
 - Utiliser les permissions et groupes Django pour gérer les rôles (patient, prestataire).
 - URL Signées pour les accès temporaires si nécessaire.
- **A02:2021-Cryptographic Failures** :
 - Utilisation exclusive de HTTPS.
 - Utilisation d'algorithmes de hachage de mots de passe modernes et salés.
 - Ne jamais stocker d'informations sensibles en texte clair.
- **A03:2021-Injection** :
 - Django ORM protège naturellement contre les injections SQL.
 - Utilisation de paramètres de requête sécurisés pour les bases de données.
- **A04:2021-Insecure Design** :
 - Révision régulière de l'architecture et du design pour identifier les vulnérabilités potentielles avant le codage.
 - Principes du moindre privilège pour les utilisateurs et les services.
- **A05:2021-Security Misconfiguration** :
 - Configuration sécurisée des serveurs, bases de données et dépendances.
 - Désactivation des modes de débogage en production.
 - Rotation régulière des clés secrètes.
 - En-têtes HTTP de sécurité (CSP, X-Frame-Options, X-Content-Type-Options).
- **A06:2021-Vulnerable and Outdated Components** :
 - Maintenance et mise à jour régulières des bibliothèques et dépendances Python, Django, JavaScript, CSS.
 - Surveillance des alertes de sécurité des dépendances.
- **A07:2021-Identification and Authentication Failures** :
 - Gestion robuste des sessions (sessions sécurisées HTTP-only).
 - Limitation des tentatives de connexion (pour prévenir la force brute).
 - Exigence de mots de passe forts.

- **A08:2021-Software and Data Integrity Failures :**
 - Validation stricte des entrées utilisateur côté client et côté serveur.
 - Vérification de l'intégrité des fichiers téléchargés.
- **A09:2021-Security Logging and Monitoring Failures :**
 - Enregistrement des événements de sécurité pertinents (tentatives de connexion, modifications de données critiques).
 - Mise en place d'alertes pour les activités suspectes.
- **A10:2021-Server-Side Request Forgery (SSRF) :**
 - Validation de toutes les URL ou ressources externes accessibles par le serveur.

7.3. Gestion des Fichiers

- **Téléchargement :** Restreindre les types de fichiers (images uniquement pour les logos/photos), limiter la taille des fichiers.
 - **Stockage :** Les fichiers téléchargés ne doivent pas être exécutables. Stocker les fichiers médias hors de la racine web (ou via un service de stockage cloud sécurisé).
-

8. Phases du Projet et Livrables

Le projet sera découpé en phases itératives.

1. Phase 1 : Initialisation et Setup

- **Livrables :**
 - Projet Django initialisé.
 - Tailwind CSS configuré et intégré.
 - Models Django (schéma de BDD) implémentés.
 - Base de code Git (dépôt).
 - Environnement de développement Docker Compose fonctionnel.

2. Phase 2 : Authentification et Profils

- **Livrables :**
 - Gestion complète de l'authentification (inscription, connexion, déconnexion, réinitialisation mdp).
 - Gestion des profils User, Patient (impliqué par User) et OrganismeDeSante (création/édition de base).
 - Permissions de base Prestataire/Patient.

3. Phase 3 : Gestion des Services (Côté Prestataire)

- **Livrables :**
 - Interfaces Prestataire pour ajouter/modifier/supprimer ServiceMedical, ActeMedical et PrestataireActe.
 - Interface pour gérer les Assurance et PriseEnChargeAssurance (PriseEnChargeActeAssurance en V2 si trop complexe).

4. Phase 4 : Recherche et Visualisation (Côté Patient)

- **Livrables :**
 - Page d'accueil avec moteur de recherche.

- Pages de liste et de détail des prestataires et de leurs services/actes.
 - Filtres de recherche (localisation, assurance, service).
 - 5. **Phase 5 : Panier et Communication**
 - **Livrables :**
 - Fonctionnalité de panier (Cart et CartItem).
 - Calcul d'estimation des coûts avec simulation d'assurance.
 - Fonctionnalité "Contacter via WhatsApp".
 - 6. **Phase 6 : Finalisation et Déploiement**
 - **Livrables :**
 - Tests complets (unitaires, intégration, fonctionnels).
 - Optimisation des performances.
 - Renforcement de la sécurité (audit, corrections).
 - Documentation technique et utilisateur simplifiée.
 - Configuration pour le déploiement en production.
-

9. Gestion des Itérations et des Retours

Chaque phase se terminera par une revue. Je vous présenterai les livrables, et votre feedback sera recueilli pour les ajustements nécessaires avant de passer à la phase suivante. Le principe est d'itérer rapidement pour s'assurer que le développement reste aligné avec votre vision.